

The STL Vector Class

CSC 240 Data Structures Fall 2025

Table of Contents

- [1. Vector constructors](#)
- [2. List of vector member functions](#)
- [3. Minimal vector API for Data Structures](#)
- [4. PRACTICE Review of basic vector operations](#)
- [5. PRACTICE Find the error and correct the code](#)
- [6. Using an iterator with vector](#)
- [7. PRACTICE Simplify an iterator](#)
- [8. Inserting new elements into a vector](#)
- [9. PRACTICE Insert a single value in a vector](#)
- [10. PRACTICE Insert a range of elements into a vector](#)

1. Vector constructors

- **Tip:** Check `std::vector` [at cppreference.com](http://cppreference.com) before and after this section to see if your understanding changed!
- `vector` has five constructors, that is five different ways of creating containers. We illustrate these with working code.

1. **Default constructor:** creates an empty vector

```
vector<string> names; // an empty vector of string objects
names.push_back("Jim");
names.push_back("Jane");
names.push_back("Joe");
for (auto name : names)
    cout << name;    // name is NOT an iterator - it is a copy of each vector element
// behind the loop an iterator std::vector<std::string>::iterator is used!
// names.begin() to names.end()
```

JimJaneJoe

2. **Fill constructor:** creates a vector of length size

```
vector<bool> values (3, 0); // a vector of three booleans (false)
for (auto elem : values) cout << boolalpha << elem << " ";
```

```
false false false
```

3. **Range constructor:** creates a vector of a range of values specified by two iterators.

```
vector<int> vec {1,2,3,4,5,6,7,8};
auto it1 = vec.begin() + 2; // 3
auto it2 = vec.end() - 2;   // 7
cout << *it1 << " " << *it2 << endl;

vector<int> vec2(it1,it2); // sub-range from it1 (3) to it2 (7)
for (auto elem : vec2) cout << elem << " ";
```

```
3 7
3 4 5 6
```

4. **Copy constructor:** creates a vector by copying another vector

```
vector<string> names = {"Jimmy", "Johnny", "Jenny"};
vector<string> names2(names); // copy of names vector
for (auto elem : names2) cout << elem << " ";
```

```
Jimmy Johnny Jenny
```

2. List of vector member functions

There are 40 (!) member functions in C++20 (including the constructors and overloaded operators). We order member functions by category:

1. Construction and assignment
2. Element access
3. Iterator support
4. Capacity operations
5. Modifiers
6. Allocator

7. Comparisons

This **function overload** is intentional: STL containers are designed for performance, generality, and type safety, favoring completeness over simplicity. C takes the opposite view: Simple over safe and complete.

1. Construction & Assignment

Function	Description
constructor	Creates vector (default, copy, move, range, init)
operator=	Assigns contents
assign(size, value)	Replaces contents from range or count+value
assign(iterator1, iterator2)	

2. Element Access

Function	Description
at(index)	Returns reference to element with bounds checking
operator[]	Returns reference to element without bounds checking
front()	Returns reference to first element
back()	Returns reference to last element
data()	Returns pointer to first element in container

3. Iterators¹

Function	Description
begin()	Iterator to first element
end()	Iterator past last element
cbegin()	Const iterator to first element

Function	Description
cend()	Const iterator past last element
rbegin()	Reverse iterator to last element
rend()	Reverse iterator before first
crbegin()	Const reverse iterator to last
crend()	Const reverse iterator before first

4. Capacity

Function	Description
empty()	Returns true if container is empty
size()	Returns number of elements
max _{size} ()	Max number of elements (theoretically) possible
capacity()	Returns number of elements that underlying array can hold, without reallocating the array ²
resize(n)	Changes number of elements to n
resize(n, value)	Changes number of elements to n - if size() < n then the new elements are initialized with value
reserve(n)	Increases capacity to n (preallocation)
shrink _{tofit} ()	Requests to reduce capacity() to be equal size()

5. Modifiers

Function	Description
clear()	Erases all elements
insert(it, n, value)	Inserts n new elements at position it

Function	Description
	with that (single) value
emplace(it, value)	Constructs element in-place at position it (new element inserted before position it)
erase(it)	Removes element at position it and returns iterator pointing to the next element
push _{back} (value)	Adds element to end
emplace _{back} (value)	Constructs element at end
pop _{back} ()	Removes last element
swap(second)	Swaps content of calling vector and of the second object (same data type)

6. Allocator

Function	Description
get _{allocator} ()	Returns allocator (custom memory management)

7. Comparisons (C++20)

Function	Description
operator==	Equality comparison
operator<=>	Three-way comparison (spaceship operator) ³

3. Minimal vector API for Data Structures

Fortunately, you don't need to memorize 40 functions - only these 10 are really critical for competently using vector containers:

	Category	Function	Purpose
1	Access	operator[]	Random access
2	Capacity	size()	Track current size
3	Capacity	reserve()	Preallocate storage
4	Capacity	capacity()	Know current storage
5	Modification	push _{back} ()	Add element to end
6	Modification	pop _{back} ()	Remove last element
7	Modification	insert()	Insert at position
8	Modification	erase()	Remove at position
9	Modification	clear()	Remove all elements
10	Iteration	begin(), end()	Traverse elements

4. **PRACTICE** Review of basic vector operations

- Read and understand the following program, run it, and then comment each section.

```
#include <iostream>
#include <vector> // ...
using namespace std; // ...

int main()
{
    // ...
    const int SIZE = 10;

    // ...
    vector<int> numbers(SIZE);

    // ...
    for (int index = 0; index < numbers.size(); index++)
        numbers[index] = index;

    // ...
    for (auto element : numbers)
        cout << element << " ";
}
```

```
    cout << endl;
    return 0;
}
```

0 1 2 3 4 5 6 7 8 9

0 1 2 3 4 5 6 7 8 9

- Solution:

```
#include <iostream>
#include <vector>    // Include vector class header file
using namespace std; // Use runtime STL std::

int main()
{
    // Define an integer constant.
    const int SIZE = 10;

    // Define a vector to hold 10 integer values.
    vector<int> numbers(SIZE);

    // Store the values 0 through 9 in the vector.
    for (int index = 0; index < numbers.size(); index++)
        numbers[index] = index;

    // Display the vector elements using a range-based loop.
    for (auto element : numbers)
        cout << element << " ";
    cout << endl;

    return 0;
}
```

0 1 2 3 4 5 6 7 8 9

5. **PRACTICE** Find the error and correct the code

1. What's the error? Can you fix it?

```
vector<int> numbers;  
numbers[0]=99;
```

Solution:

```
vector<int> numbers = {0};  
numbers[0]=99; // vector must be non-empty
```

2. What's the error? Can you fix it?

```
vector<int> numbers;  
push_back(10);
```

Solution:

```
vector<int> numbers;  
numbers.push_back(10); // push_back declared in numbers scope
```

3. What's the error? Can you fix it?

```
vector<bool> numbers;  
numbers.push_back(true);  
numbers.pop_back(false);
```

Solution:

```
vector<bool> truth;  
truth.push_back(true); // add `true` at end of truth (truth[0])  
truth.pop_back();      // pop_back() has no arguments
```

4. What's the error? Can you fix it?

```
vector<string> names = {"Jimmy Dean", "Johnny Cash", "Jack Black"};  
cout << names.at(3) << endl;
```

Solution:

```
vector<string> names = {"Jimmy Dean", "Johnny Cash", "Jack Black"};
cout << names.at(2) << endl; // at(3) throws an out_of_range exception
```

Jack Black

5. What's the error? Can you fix it?

```
vector<string> names = {"Jimmy Dean", "Johnny Cash", "Jack Black"};
vector<string> names2;
names2[0] = names[0];
names2[1] = names[1];
names2[2] = names[2];
```

Solution:

```
vector<string> names = {"Jimmy Dean", "Johnny Cash", "Jack Black"};
vector<string> names2;
// names2[0] = names[0]; // cannot assign to names2 before it exists
names2 = names; // overloaded assignment operator= for vector
for (auto element : names2)
    cout << element << " ";
```

Jimmy Dean Johnny Cash Jack Black

6. Using an iterator with vector

- The vector class supports the use of these iterators:
 1. iterator (forward iterator)
 2. const_iterator (constant iterator)
 3. reverse_iterator (backward iterator)
 4. const_reverse_iterator (constant backward iterator)
- Compare the member support functions for getting iterators: begin(), end(), cbegin(), cend(), rbegin(), rend(), crbegin(), crend().
- You can simplify the definition of an iterator with auto. In the following code block loop 1 and loop 2 are equivalent:

```
vector<int> num {1,1,1,1,1};
```

```
// loop 1
vector<int>::iterator it;
for (it = num.begin(); it != num.end(); it++) {cout<<*it;}

cout << endl;
// loop 2
for (auto it = num.begin(); it != num.end(); it++) {cout<<*it;}
```

```
11111
11111
```

- Where does it point? How many times does the loop run?

Since num is empty, num.begin() equals num.end() and the loop never runs. it does not point to any valid element. No memory is reserved for num until elements are inserted or reserve(n) is called.

7. **PRACTICE** Simplify an iterator

- Create a vector that holds these names: Joe, Jim, Jack, Jane.
- Define a forward iterator it using the iterator keyword.
- Use it to display each element in the vector.
- When the program works, use another iterator it2 using the auto keyword to display the elements again without explicitly defining the iterator, but in reverse order, using rbegin() and rend()!
- Hint: rbegin() begins at the reverse beginning (the end) of the vector, and rend() ends at its reverse end (the beginning).
- Sample output:

```
#+RESULTS:
: Joe Jim Jack Jane
: Jane Jack Jim Joe
```

- Solution:

```
vector<string> names = {"Joe", "Jim", "Jack", "Jane"};

vector<string>::iterator it;

for(it = names.begin(); it != names.end(); it++)
    cout << *it << " ";
```

```
cout << endl;

for(auto it2 = names.rbegin(); it2 != names.rend(); it2++)
    cout << *it2 << " ";
```

```
Joe Jim Jack Jane
Jane Jack Jim Joe
```

8. Inserting new elements into a vector

- To insert one or more new elements at a specified position in a vector, use `insert`. The full definition is complicated (see cppreference.com)!
- There are multiple overloads (alternative uses) but these three are most common:

1. Insert a single value of type `T` at the specified position:

```
iterator insert(iterator pos, const T& value);
```

2. Insert count copies of a value:

```
iterator insert(iterator pos, size_type count, const T& value);
```

3. Insert a range of values:

```
iterator insert(iterator pos, iterator pos1, iterator pos2);
```

- The iterator is like a bookmark inside the vector. Insertion happens before the iterator position.
- Example: Inserting a single value in a numeric vector

```
vector<int> v{1,2,3,4,5};
for (auto element : v) cout << element << " "; cout << endl; // print vector
auto it = v.begin() + 2; // points to element 4
v.insert(it, 3);          // insert 3 before 4
for (auto element : v) cout << element << " ";
```

```
1 2 3 4 5
1 2 3 3 4 5
```

- The function returns an iterator to the newly inserted element:

```
vector<int> v {1,2,3,4,5};
auto inserted = v.insert(v.begin(), 99);
cout << *inserted << endl;
```

```
99
```

- Example: Insert multiple copies

```
vector<int> v {1,2,5};
v.insert(v.begin() + 2, 2, 3); // insert two 3s before 5
for (auto element : v) cout << element << " ";
```

```
1 2 3 3 5
```

- Example: Insert a range of values:

```
vector<int> a{10,20};
vector<int> b{1,2,3};
a.insert(a.begin() + 1, b.begin(), b.end()); // insert b before 20
for (auto element : a) cout << element << " ";
```

```
10 1 2 3 20
```

- Does insert modify a vector in-place or does it create a copy?
 - The vector is modified in-place (you're not getting a new copy) but memory management does take place:
 - If the vector does not have enough capacity, a new, larger array is allocated, all elements are moved to the new memory, the old array is deleted, and all iterators, pointers, references become invalid.
- **Challenge:** What's the final vector in the following example? Guess the result before trying this yourself!

```
vector<char> letters {'A', 'C', 'D'};  
letters.insert(letters.begin() + 1, 'B');  
letters.insert(letters.end(), {'E', 'F', 'G'});  
  
for (char letter : letters) cout << letter << " ";
```

A B C D E F G

9. **PRACTICE** Insert a single value in a vector

- Use the starter code below to
 1. Define an integer vector numbers with the numbers 1, 2, 3, 4, 5.
 2. Define an iterator it pointing to the second element of numbers.
 3. Insert a new element with the value 99.
 4. Display the vector elements.
- Sample output:

1 99 2 3 4 5

- Starter code: <https://tinyurl.com/insert-value>

```
#include <iostream>  
#include <vector>  
using namespace std;  
  
int main()  
{  
    // Define a vector `numbers` with 5 integer values 1,2,3,4,5.  
  
    // Define an iterator `it` pointing to the second element.  
  
    // Insert a new element with the value 99  
  
    // Display the vector elements.  
  
    return 0;  
}
```

- Starter code: <https://tinyurl.com/insert-range>

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define a vector `numbers` with 5 integer values 1,2,3,4,5.
    vector<int> numbers = {1,2,3,4,5};
    // Define an iterator `it` pointing to the second element.
    auto it = numbers.begin() + 1;
    // Insert a new element with the value 99
    numbers.insert(it,99);
    // Display the vector elements.
    for (int num : numbers) cout << num << " ";
    return 0;
}
```

1 99 2 3 4 5

- Modify the program to insert 4 numbers 99 before the 3rd element of the original vector. Sample output: 1 2 99 99 99 99 3 4 5.
- Solution:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define a vector `numbers` with 5 integer values 1,2,3,4,5.
    vector<int> numbers = {1,2,3,4,5};
    // Define an iterator `it` pointing to the second element.
    auto it = numbers.begin() + 2;
    // Insert a new element with the value 99
    numbers.insert(it,4,99);
    // Display the vector elements.
    for (int num : numbers) cout << num << " ";
    return 0;
}
```

```
1 2 99 99 99 99 3 4 5
```

10. **PRACTICE** Insert a range of elements into a vector

- Use the starter code below to
 1. Define **two** integer vectors v1 and v2 with the elements {1,2,3} and the elements {100,200,300,400,500}, respectively.
 2. Define **three** iterators it1, it2, and it3:
 - it1 points at 2 in v1
 - it2 points at 100 in v2
 - it3 points at 400 in v2.
 3. Insert the range of elements between it2 and it3 into v1
 4. Display the elements of v1

- Sample output:

```
1 100 200 300 2 3
```

- Starter code: <https://tinyurl.com/insert-range>

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define two vectors v1, v2.

    // Define three iterators it1, it2, it3.

    // Insert a range of elements into v1.

    // Display the elements of v1.

    return 0;
}
```

- Solution:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define two vectors v1, v2.
    vector<int> v1{1,2,3};
    vector<int> v2{100,200,300,400,500};

    // Define three iterators it1, it2, it3.
    auto it1 = v1.begin() + 1;
    auto it2 = v2.begin();
    auto it3 = v2.begin() + 3;

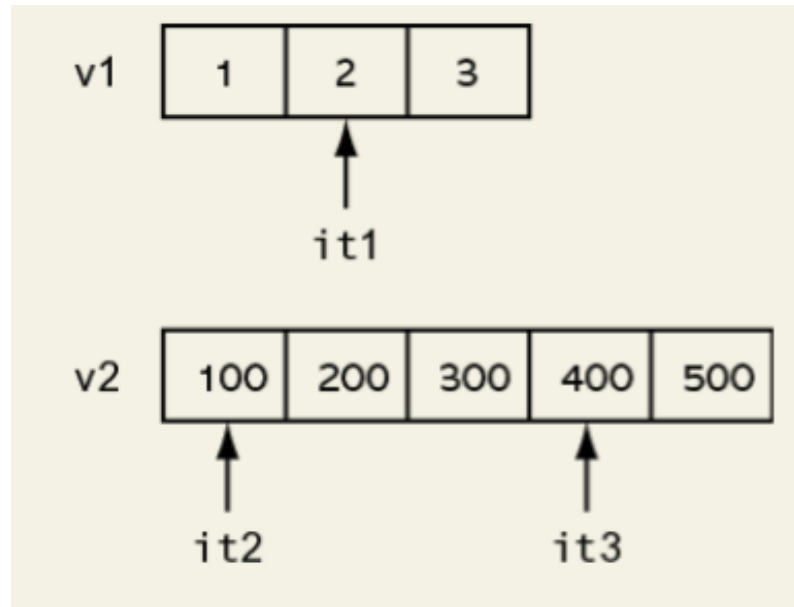
    // Insert a range of elements into v1.
    v1.insert(it1, it2, it3);

    // Display the elements of v1.
    for (auto num : v1) cout << num << " ";

    return 0;
}
```

```
1 100 200 300 2 3
```

- Visualization:



Footnotes:

¹ `const` iterator allows read-only access to container elements. It cannot be used to modify elements it points to.

² For example, after `vec.reserve(100)`, the vector will not reallocate until more than 100 elements are added.

³ Instead of writing all six relational operators `<`, `<=`, `>`, `>=`, `==`, `!=`, you write `<=>` and the compiler generates the rest.

Author: Marcus Birkenkrahe

Created: 2025-11-05 Wed 20:55